Week 2 Lecture Notes: Classes, `__init__` and Inheritance

One interface for every sensor on the vehicle

AI for Aerospace & Mechanical Engineering (PhD), Block 0

> **Provenance:** new content. The instructor's PDFs do not cover Python OOP. The Python material is rewritten (not copied) from W3Schools (Python Classes, `__init__()` Method, Class Properties, Inheritance, Polymorphism) and GeeksforGeeks (Class and Instance Variables, `super()`, `property()`, Abstract Classes in Python, Understanding Python Dataclasses, Getting Started with Pytest). The ISA pressure-altitude model is adapted from J. D. Anderson, *Introduction to Flight*, Ch. 3; the flat-Earth local frame from P. D. Groves, *Principles of GNSS, Inertial, and Multisensor Integrated Navigation Systems*, Ch. 2.

1. The problem

Our test vehicle carries three sensors. An **IMU** produces six numbers at 50 Hz (specific force in m/s², angular rate in deg/s); it has constant biases and saturates at its full-scale range. A 16-beam **LiDAR** produces ranges at 10 Hz and reports 0.0 or `inf` when a beam gets no echo. A **GPS** receiver produces latitude, longitude, altitude and HDOP at 5 Hz, and some fixes are garbage. Every later part of the course (lane detection, obstacle detection, the controller, the final project) consumes these streams.

The procedural way to handle them is one function per job (`validate`, `preprocess`, `to_frame`, `plot`) with an `if/elif` chain on the sensor type inside each. It fails in four ways, all shown live in the lab: a misspelled type (`"IMU"` for `"imu"`) falls through the chain and returns biased data **without any error**; a sample with the wrong number of values is accepted because `zip` stops early; configuration such as biases ends up in globals, so two IMUs with different biases are impossible; and adding a barometer means editing every function that contains the chain. Object-oriented programming fixes all four by bundling each sensor's data with its behaviour and putting every sensor behind one shared interface.

2. Classes, objects, `__init__` and `self`

A **class** is a blueprint; an **object** (instance) is one thing built from it. The special method `__init__` runs automatically when an object is created and stores its attributes on `self`, the object being built. A **method** is a function defined inside the class; when you call `baro.altitude_m(95_000.0)`, Python passes `baro` as the first argument, `self`.

Worked Example 1: ISA pressure altitude

In the troposphere the International Standard Atmosphere relates static pressure p to altitude:

$$h = \frac{T_0}{L}\left[1 - \left(\frac{p}{p_{\text{ref}}}\right)^{RL/g_0}\right], \qquad \frac{T_0}{L} = \frac{288.15}{0.0065} = 44\,330.8\ \text{m}, \qquad \frac{RL}{g_0} = \frac{287.053 \times 0.0065}{9.80665} = 0.190263.$$

For $p = 95\,000$ Pa and the standard $p_{\text{ref}} = 101\,325$ Pa: $p/p_{\text{ref}} = 0.937577$, $\ln(0.937577) = -0.064457$, and $0.190263 \times (-0.064457) = -0.012264$. Hence $(p/p_{\text{ref}})^{0.190263} = e^{-0.012264} = 0.987811$ and

$$h = 44\,330.8 \times (1 - 0.987811) = 44\,330.8 \times 0.012189 = 540.3 \text{ m}.$$

A `SimpleBarometer` class stores `sensor_id` and `reference_pressure_pa` in `__init__` and implements this formula in a method. A second object built with the airfield QNH of 100 500 Pa returns 472.2 m for the same pressure: one class, two objects, different state.

3. Class attributes, instance attributes and properties

An attribute assigned in the class body belongs to the **class** and is shared by every object; one assigned on `self` belongs to **one object**. Shared is right for constants (every IMU has the same channel names) and wrong for state. The classic bug is a class-level `readings = []`: `self.readings.append(x)` finds the shared list and mutates it, so every sensor sees every other sensor's samples. Assigning `obj.unit = "rad/s"`, by contrast, creates a new instance attribute that *shadows* the class attribute. Rule: **mutable state is created in `__init__`**.

A **property** looks like an attribute but runs a method. Its *setter* can validate: `imu.rate_hz = 0` raises `ValueError` before a zero ever reaches a division. A property without a setter is **read-only** (a sensor's ID must never change), and a computed property such as `period_s = 1 / rate_hz` can never go stale. The value itself is kept in an attribute with a leading underscore (`_rate_hz`), the Python convention for "internal".

4. Inheritance, `super()` and the method resolution order

`class Barometer(Sensor)` **inherits** every method and property of `Sensor` and may add new ones or **override** existing ones (same name, new body). When the child needs its own `__init__`, it must still let the parent initialise the parent's part: `super().__init__(sensor_id, rate_hz)`. Python searches for a method along the **method resolution order** (MRO), e.g. `Barometer → Sensor → RatedSensor → object`. If a child forgets `super().__init__`, construction still succeeds but the parent's attributes never exist; the error (`AttributeError`) appears much later, far from its cause. In-class Problem 2 follows a three-level chain by hand.

5. Abstract base classes and dataclasses

A plain base class with a do-nothing `preprocess` lets a subclass that forgot (or misspelled) the method run silently. An **abstract base class** turns the interface into a contract: `class BaseSensor(ABC)` declares `validate` and `preprocess` with `@abstractmethod`, and Python refuses to instantiate `BaseSensor` itself or any subclass that has not implemented both. The error happens at construction, on the bench, not in flight.

Each stored sample is a `SensorReading(timestamp, values, sensor_id, is_valid)`. Written by hand the record needs `__init__`, `__repr__` and `__eq__`; `@dataclass` generates all three from the annotated fields. `frozen=True` makes a reading immutable (a logged measurement must never change) and hashable; `slots=True` removes the per-object `__dict__`, which matters for 100 000

readings. `values` is a `tuple`, not a NumPy array, because `array == array` returns an array and breaks the generated `__eq__` and `hash`.

6. The course hierarchy and the template method

`src/aerospace_ai/sensors.py` combines all of the above. `BaseSensor` owns everything that is the same for every sensor: ID, validated rate, a bounded buffer, conversion to arrays and DataFrames, and the method `add_reading`, which runs a fixed algorithm: check the shape, check that time increases, call `validate(raw)`, call `preprocess(raw)` if valid, and store a `SensorReading`. Steps 3 and 4 are the only ones a subclass writes. This is the **template method** pattern. Invalid samples are stored unprocessed with `is_valid=False` for diagnosis and excluded from `to_array()`.

Worked Example 2: IMU bias removal and saturation

An IMU set to ±4 g has full scale $4 \times 9.80665 = 39.23$ m/s². Calibration gave accelerometer bias $(0.10,\ -0.05, 0.20)$ m/s² and gyro bias $(0.8,\ -0.3, 0.1)$ deg/s.

- Raw $(0.35,\ -0.05, 10.05 \mid 0.8,\ -0.3, 2.1)$ becomes $(0.25, 0, 9.85 \mid 0, 0, 2.0)$, and $n = \sqrt{0.25^2 + 9.85^2}/g_0 = 9.8532/9.80665 = 1.005$.
- Raw $(0.10, \mathrm{NaN}, 9.90 \mid \ldots)$ is invalid (not finite) and stored raw.
- Raw $a_z = 39.50 \geq 39.23$ is **saturated**: the sensor clipped, the true value is at least 4 g and unknown, so the sample is invalid.
- Raw $(1.30,\ -0.45, 19.80 \mid 0.8, 4.7, 0.1)$ becomes $(1.2,\ -0.4, 19.6 \mid 0, 5.0, 0)$, and $n = 19.641/9.80665 = 2.003$, Week 1's example.

`IMUSensor.calibrate` estimates both biases from parked data, where the true specific force is $(0, 0, +g_0)$ and the true rate is zero. Applied to the Week 1 flight log with a ±4 g range, the three +40 m/s² spikes hit full scale and are rejected; the peak load factor of the valid samples is **1.81 g**, below the 2.0 g limit. In Week 1 the same answer needed a rolling median.

7. Polymorphism, the Assignment 1 sensors and testing

Polymorphism means one call, type-specific behaviour: `for s in suite: s.add_reading(t, raw)` works for any mix of sensors with no `if/elif` on types, so a new `ThermometerSensor` joins with zero changes to the loop.

The **LiDAR** overrides the `channel_names` property (its channel count depends on `n_beams`); beam i points at $-180° + i \cdot 22.5°$. A beam is usable if its range is finite and within $[0.15, 12]$ m; unusable beams become NaN, never zero, because a zero range would read as an obstacle touching the robot. A scan is valid if at least half its beams are usable.

Worked Example 3: GPS to local metres

With $R = 6\,371$ km, one degree of latitude is $R\pi/180 = 111\,195$ m everywhere; one degree of longitude is $111\,195 \cos\varphi_0$. At Mehrabad ($\varphi_0 = 35.6892°$, $\cos\varphi_0 = 0.81221$) that is 90 314 m. A fix 0.001° north and 0.001° east of the origin is therefore at $(e, n) = (90.3, 111.2)$ m:

$$n = R\,\Delta\varphi, \qquad e = R\cos\varphi_0\,\Delta\lambda, \qquad u = h - h_0 .$$

The error of this flat-Earth approximation is below 1 cm over the 80 m test track. Fixes with HDOP above 5, NaN coordinates or $|\text{lat}| > 90°$ are rejected.

Testing. A `pytest` test is a `test_*` function with plain `assert`. `pytest.raises` checks that an error happens, `@pytest.mark.parametrize` runs one test on many inputs, and a `@pytest.fixture` builds a fresh object per test. Good tests check **numbers**: Assignment 1 runs each student's suite against three subtly broken implementations (east without $\cos\varphi_0$, HDOP limit ignored, out-of-range echoes kept) and rewards each one caught.

What we now know how to do

- Bundle data and behaviour in classes; use `__init__` and `self` correctly.
- Keep mutable state per object; guard attributes with validated and read-only properties.
- Reuse code with inheritance and `super()`; enforce an interface with an abstract base class.
- Write immutable records with `@dataclass(frozen=True, slots=True)`.
- Process a mixed list of sensors polymorphically and test the hierarchy with pytest.

Practice: in-class problems `in_class_problems/week02_problems.ipynb`, lab `notebooks/week02/week02_oop_sensors.ipynb`, and Assignment 1 (`assignments/assignment01/`).